

Lab Report – EE-170L Computer Programming (Lab 10)

NAME: _MAAZ WAHAB _____

REG NO: _BF25NWELE0719 _____

SEMESTER: 2nd

LAB: 10 (Programming)

DATE: __6 may 2026_____

1. Objective

The main goal of this lab is to practice **advanced concepts of functions in C++**. Functions are central to modular programming, and this session focuses on:

- **Function Call by Value**
- **Function Call by Reference**
- **Default Values in Parameters**

By the end of this lab, students should be able to:

- Differentiate between call by value and call by reference.
- Write functions that modify variables directly using references.
- Use default parameter values to simplify function calls.

2. Theory

2.1 Call by Value

In **call by value**, a copy of the variable is passed to the function. Any changes made inside the function do not affect the original variable.

2.2 Call by Reference

In **call by reference**, the function works directly with the memory address of the variable. Changes made inside the function affect the original variable.

2.3 Default Values in Parameters

Default values allow functions to be called with fewer arguments. If no value is provided, the default is used.

3. Programs and Outputs

Task 1: Function Call by Value

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
// Function prototype
```

```
void func(int);
```

```
int main() {
```

```
    int i = 10;
```

```
    func(i); // Call by value
```

```
    cout << "The value of i remains " << i << endl;
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
void func(int i) {  
    i = i + 10; // Change local copy  
    cout << "Inside function, the value of i changed to " << i << endl;  
}
```

Output:

Code

Inside function, the value of i changed to 20

The value of i remains 10

Task 2: Function Call by Reference

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
// Function prototype
```

```
void swapping(int &, int &);
```

```
int main() {
```

```
    int x = 50, y = 70;
```

```
    cout << "Before calling function: x = " << x << " and y = " << y << endl;
```

```
    swapping(x, y); // Call by reference
```

```
    cout << "After calling function: x = " << x << " and y = " << y << endl;
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```

void swapping(int &x1, int &y1) {

    int temp = x1;

    x1 = y1;

    y1 = temp;

}

```

Output:

Code

Before calling function: x = 50 and y = 70

After calling function: x = 70 and y = 50

Task 3: Default Values in Parameters

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
// Function definition with default parameters
```

```

int divide(int a = 8, int b = 2) {

    return a / b;

}

```

```

int main() {

    cout << divide() << endl;    // Uses both default values

    cout << divide(12) << endl;  // Uses default for b

    cout << divide(63, 7) << endl; // Uses both provided values

    return 0;

}

```

Output:

Code

4

6

9

Lab Tasks

Task 1: Function with Parameters

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
// Function prototype
```

```
void multiply(int num1, int num2);
```

```
int main() {
```

```
    multiply(5, 7); // Call function with two integers
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
void multiply(int num1, int num2) {
```

```
    int product = num1 * num2;
```

```
    cout << "Product = " << product << endl;
```

```
}
```

Output:

Code

Product = 35

Task 2: Function with Return Value

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
// Function prototype
```

```
int getSquare(int number);
```

```
int main() {
```

```
    int result = getSquare(9); // Call function
```

```
    cout << "Square = " << result << endl;
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
int getSquare(int number) {
```

```
    return number * number;
```

```
}
```

Output:

Code

Square = 81

4. Discussion

This lab demonstrated how functions can be used more effectively:

- **Call by value** keeps the original variable safe from modification.
- **Call by reference** allows direct manipulation of variables, useful for swapping or updating values.
- **Default parameters** simplify function calls by reducing the number of arguments needed.

By practicing these techniques, I learned how to control variable scope, memory usage, and function flexibility.

5. Conclusion

In this lab, I wrote programs using **call by value, call by reference, and default parameters**. I learned how each method works and where it can be applied.

- Call by value is safe but limited.
- Call by reference is powerful for modifying variables directly.
- Default parameters make functions more versatile.

This lab improved my understanding of advanced function concepts and helped me practice writing efficient, reusable code.